

Getränkebestellungen am Rattenfest

Studienarbeit
Herbstsemester 2026

Version: Lokaler Build
Datum: 01.01.1980

Team: Andrin Klarer (andrin.klarer@ost.ch)
Jasmin Fässler (jasmin.faessler@ost.ch)
Betreuer: Frieder Loch (frieder.loch@ost.ch)

Departement Informatik
Ostschweizer Fachhochschule



0 Inhaltsverzeichnis

1	Einleitung	3
1.1	Problemstellung	3
1.2	Vorhandene Lösungswege	4
1.3	Anwendungsszenarien (detaillierter)	4
1.4	Erfolgszenario	4
2	Theoretischer Hintergrund	5
2.1	Recherche User Experience während Rattenfest	5
3	Methode	5
3.1	Prototyp / Skizze der Lösung	5
3.2	UI und Backend Konzept (Software-Architektur)	5
4	Ergebnisse	5
5	Diskussion inkl. Limitationen	5
6	Fazit und Ausblick	5
7	Projektinformationen	6
7.1	Projektplan	6
7.2	Langfristiger Zeitplan	7
7.3	Kurzfristiger Plan	8
7.4	Meeting Notizen	8
8	Bilderverzeichnis	9
9	Tabellenverzeichnis	9
10	Bibliografie	9

1 Einleitung

1.1 Problemstellung

Digitalisierung der Getränkeverwaltung während dem Rattenfest zur Effizienzsteigerung. Getränke werden zwischen dem Zentrallager (Kühlwagen) und den Verkaufsstellen (Bars, Backstage) transportiert. Bars (Bar Mitarbeiter) bestellen die Getränke, die Rattenfest Arbeiter stellen die Getränke bereit und die Bar Mitarbeiter holen die Getränke ab.

Vorher mussten Bar Mitarbeiter zum Kühlwagen, ihre Bestellung aufgeben, warten und diese zurücktransportieren. Während dem Rattenfest gibt eine Bar ca. ... Bestellungen auf. Es wurde bereits ein einfacher Prototyp erstellt und getestet. Vor dem Rattenfest erstellen die Bars eine Schätzung an benötigten Getränken, damit das Rattenfest OK offensichtliche Fehler korrigiert und Getränke vorbestellt. Dies wurde bisher mit einem Excel Sheet gelöst.

Ziel: gute Website mit guter Architektur die lange verwendet werden kann und ihren Zweck erfüllt.
Lösung massgeschneidert für das Rattenfest.

1.1.1 These

1.1.2 Anforderungen

Es wird eine Lösung gesucht, die einfach nutzbar ist (keine versteckten Buttons, suchen von Funktionen..) und während dem Rattenfest muss es fehlerfrei laufen (Redundanz, Alternative Serverlösungen, ..) weitere Anforderungen (noch priorisieren) :

- Bedienbarkeit:
 - Schritte (sinnvoll) rückgängig machen können, Bilder nutzen
 - weniger Klicks und scrolls oder User nicht überfordern? -> Bar User definieren, z.B. wenn es pro Bar eine verantwortliche Person gibt darf die Bedienung schwieriger sein, damit Zeit gespart werden kann
- wartbarkeit:
 - OK Mitglieder die nicht Informatiker sind, sollten das System einfach updaten können (aber evt. Anleitung erstellen anstatt alles automatisieren) > einfacher geworden mit KI
 - Software best practices anwenden (z.B. Tests)
 - das Meiste soll in einem GUI gelöst werden können, DB Anpassungen nicht per SQL/Table Editor nötig.
- erweiterbarkeit: weitere ineffiziente Prozessabläufe vom Rattenfest digitalisieren/automatisieren ist nicht wichtig, soll möglich sein aber kann auch separat umgesetzt werden. Kommunikation im OK, Website für das Rattenfest, Ticketsystem und weiteres bleibt getrennt.
 - Code Komplexität oder Erweiterbarkeit wichtiger? Zukunftspläne?
- langfristige Lösung: natürlich sollen neue Getränke erfasst werden können, aber auch jährliche Zahlen speichern für Analysen in Zukunft und erweiterbarkeit durch weitere Tools?
- Performance:
 - sollte nicht zu teuer sein, wenn es nur am Rattenfest intensiv genutzt wird, bei ein paar Sekunden längerer Verarbeitungszeit nicht schlimm?
 - das initiale UI und Klicks sollten sofort erkannt werden und sinnvolles Feedback gezeigt werden (lade-animation, Erfolgsmeldung, Überlegung PWA)
 - Internetverbindung während Rattenfest wichtig
 - nicht so langsam, wie ein Webshop sich leisten kann
- Fehler einfach beheben:
 - Hinweise anzeigen, bei ungewöhnlichen Bestellungen (eine 0 zuviel bei Anzahl Getränke, Feld leer gelassen welches bei diesem Getränk sonst eingetragen wird, besonders kleine Bestellung..)
 - Rückgängig/Wiederholen Button genügend gross und schnell (gutes System für History finden)

- Zuverlässigkeit : sehr wichtig, keine halb verarbeiteten Bestellungen, keine falschen Informationen anzeigen
- Umstellung auf Digital verbessern
 - Pausenfunktion einbinden für RF Arbeiter, da jetzt Meldungen sofort reinkommen
 - Bestellung per Sprache erstellen (eher nicht, es ist laut)
 - Bilder nutzen, damit Bars ihre Bestellung besser einschätzen können: bei grosser Bestellung ein Bild mit vielen Getränken und bei wenigen Getränken Bild mit wenig Getränken
- Analyse/Statistics: was wäre wichtig? Export als Excel/CSV der Daten (wie? wo speichern?)
Vergleiche zwischen verschiedenen Jahren. Bessere Schätzungen
- Getränke im Lager erfassen/anpassen (RF, im voraus)
 - sinnvolle Optionen/Felder
- Bar User erstellen und teilen (RF)
 - Login ... (Bar)
- Getränke bestellen (Bar)
- Undo/Redo (siehe oben)
- History > alte Bestellungen betrachten, prüfen
- Abholung > Identität verifizieren nötig? Abholung bestätigen
- Kontaktfunktion (allgemeine Probleme/Fragen melden, Notfallmeldungen während Fest, wer von RF behandelt Meldungen)

1.1.3 Risiken und Unsicherheiten

Spezielle Umgebung: während Rattenfest

- ausserhalb von der Website: Tipps für Bars, wie sie das Gerät mit der Website einrichten können, damit es möglichst integriert und nicht lästig ist (Gerät, Halterung, Rolle zuweisen ?)
- Bedienung von Website (User Experience) eng, dunkel, laut, Bedienung eingeschränkt

Risiko eine Funktion zu vergessen: das könnte zu Tricks und Umwegen führen, deshalb User Tests machen, Zielgruppe befragen

1.2 Vorhandene Lösungswege

- event organisations softwares > meiste haben nicht fokus Getränkbestellung sondern tickets, zeitmanagement..
- Getränkelieferanten/Webshop Lösungen (Ziel: wenig Performance, es läuft das ganze Jahr durch)
- Lösungen ohne Website?
- der Vorteil gegenüber einem Chat wo man Bestellungen schreibt ist offensichtlich, aber freie Textnachrichten haben auch Vorteile > Bemerkungen-Möglichkeit bei Bestellung einbauen?

1.3 Anwendungsszenarien (detaillierter)

1.4 Erfolgszenario

Die Arbeit war erfolgreich, wenn ...

2 Theoretischer Hintergrund

2.1 Recherche User Experience während Rattenfest

2.1.1 Definition

2.1.2 Erkenntnisse

2.1.3 wichtige Probleme

2.1.4 Konkrete Massnahmen

2.1.4.1 Design

2.1.4.2 Software-Architektur

3 Methode

3.1 Prototyp / Skizze der Lösung

3.2 UI und Backend Konzept (Software-Architektur)

4 Ergebnisse

5 Diskussion inkl. Limitationen

6 Fazit und Ausblick

7 Projektinformationen

7.1 Projektplan

Der Aufwand beträgt 2 Arbeitstage pro Woche für ca. 17h/W , insgesamt 240 Stunden Aufwand.

7.1.1 Meilensteine

7.1.1.1 M1: Recherche + Konzept fertig

7.1.1.2 M2: Programmierung abgeschlossen

7.1.1.3 M3: Dokumentation bereit zum Korrekturlesen

7.2 Langfristiger Zeitplan

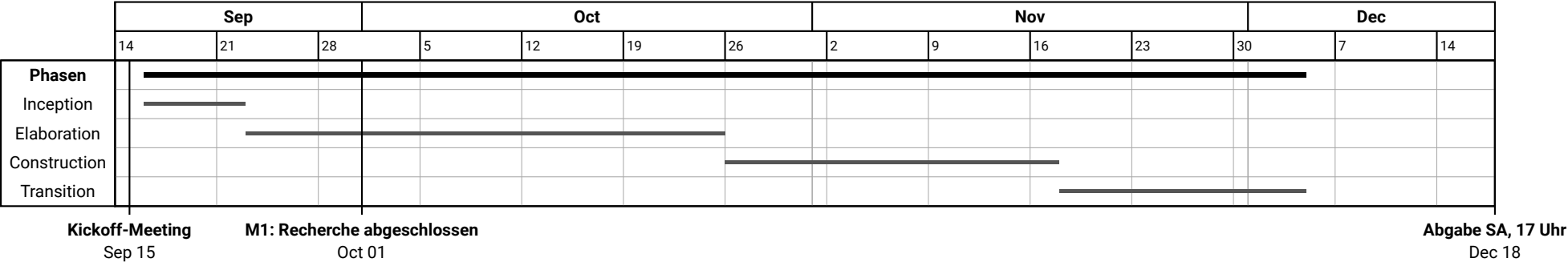


Abbildung 1: Langfristiger Zeitplan

7.3 Kurzfristiger Plan

Punkte die erledigt werden pro Woche, Struktur und ausführlichkeit kann sich ändern.

Woche 1:

- Kickoff
- Dokumentationsstruktur aufbauen + Link zu PDF
- Aufgabe konkretisieren/Arbeit grob planen
- Erfolgszenario definieren

offene Fragen: TODOs im typst

Woche 2:

-

7.3.1 Kanban Board

Backlog (2)

Dokumentationsstruktur anpassen/
ergänzen + Requirements genauer
definieren

doc A

Erfolgszenario definieren

doc ?

In Arbeit (0)

Done (1)

Typst aufsetzen,
Dokumentationsstruktur erstellen und
erstes Brainstorming

doc J

7.4 Meeting Notizen

Meetingnotizen werden separat erstellt, hier werden nur wichtige Entscheidungen notiert.

- Wöchentliches Meeting am Dienstag mit Betreuer

8 Bilderverzeichnis

Abbildung 1 Langfristiger Zeitplan 7

9 Tabellenverzeichnis

10 Bibliografie